UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Music Artists & Albums Public Perception

Systems Engineering Curriculum
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia

Course: Programming for Data Analysis

Professor: Carlos Andres Sierra Virguez

Members:

Carlos Andres Celis Herrera	20222020051
Juan Diego Lozada Gonzalez	20222020014
Cristian Santiago Lopez Cadena	20222020027

March 21, 2026

Contents

1 Executive Summary

For this second installment and the continuation of the project's development, we present the technical design of a functional and automated data engineering ecosystem. Focused on the Medallion architecture approach, which organizes data into a layered quality hierarchy: Bronze (raw, immutable JSON data), Silver (cleaned, transformed, and structured Parquet files), and Gold (highly refined data and business model). The system is designed to manage data ingestion from multiple sources by integrating semi-structured data from the Last.fm API and unstructured text data from Reddit, ensuring a scalable foundation for sentiment analysis and trend discovery.

1.1 Stack Decisions

First, to ensure portability and consistency across different development environments, the entire stack is containerized using Docker. This allows us to set up an environment without any inconsistencies, using the same set of libraries and dependencies on every machine where the project is run. However, the following key technical tools were used for the development of the project.

1. **Apache Airflow:** It was selected as a workflow orchestrator due to its robust TaskFlow API and its ability to manage complex dependencies using directed acyclic graphs (DAGs). This enables the creation of periodic processes and bridges between different layers through the use of DAGs.

2. **PostgreSql:** It serves as a backend metadata database for Airflow, ensuring the persistence of processes. This is because Airflow needs to persist the state of each run.

3. **Pandas:** It was selected as the tool needed to clean and transform the data ingested into the Bronze tier. Additionally, Apache Parquet was chosen as the primary storage format for the Silver tier. Parquet's columnar storage significantly reduces disk I/O and improves query performance compared to traditional CSV or JSON formats, which is essential for the upcoming phases.

1.2 Pipeline Design

The data processing workflow is currently divided into two stages, both of which are largely automated:

- Ingestion phase (Bronze): Built using a specific DAG, it extracts data from Last.fm. It applies a strict naming convention based on timestamps (source `_YYYYMMDD_HHMMSS.json`) to maintain a historical record of all raw data retrievals.

- Processing Phase (Silver): This stage is triggered by an Airflow FileSensor, which monitors the Bronze directory. Once a new file is detected, the pipeline runs a cleaning sequence that handles missing values, normalizes data types (for example: listeners and play counts), and applies natural language processing (NLP) techniques to textual data before persisting the result as a Parquet file.

2 Architecture Diagram

The architectural design implemented for the project is based on the Medallion Architecture pattern, divided into two clearly defined physical zones: the Local Filesystem as the persistence layer, which hosts the Bronze, Silver, and Gold layers, and the Workflow Orchestration as the processing layer. See Figure 1. Both zones communicate via unidirectional data flows, ensuring a separation of responsibilities between storage and execution.

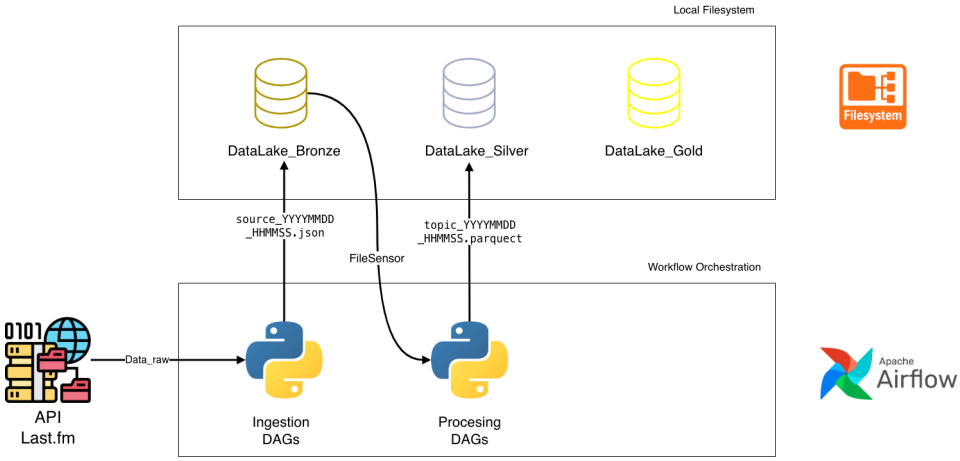


Figure 1: Architecture Medallion

2.1 Zone 1 — Local Filesystem

This zone acts as the host's file system, mounted as a Docker volume. It contains three layers of progressive storage:

- **DataLake_Bronze:** This layer receives raw data directly from the ingestion DAG. It is the only layer that writes data outside the system; the other layers only read from the previous layer and write to the next. Files are named using the format `source_YYYYMMDD_HHMMSS.json` where the `source` prefix identifies the source (Last.fm or Reddit) and the timestamp ensures uniqueness per execution, making the pipeline naturally idempotent.

- **DataLake_Silver:** The layer receives processed files from the processing DAG with the filename format `topic_YYYYMMDD_HHMMSS.parquet`. The switch from JSON to Parquet is an important technical decision: Parquet uses columnar encoding, which significantly reduces disk space requirements and speeds up the analytical reads that PySpark will perform in the Gold layer.

- **DataLake_Gold:** The Gold layer is the final stage of the Medallion Architecture and represents the data consumption zone. Unlike the previous layers, this layer does not contain raw or simply cleaned data, but rather stores highly curated and aggregated data.

2.2 Zone 2 - Workflow Orchestration

This section encapsulates the entire execution logic orchestrated by Airflow. It contains two DAGs with completely separate responsibilities:

- **Ingestion DAG:** This script receives the `Data_raw` stream from the Last.fm API, processed using Python, and stored in DataLake_Bronze. This DAG runs according to a defined periodic schedule (@daily or cron expression) and serves as the sole entry point for external data into the system. Designed using the TaskFlow API, it integrates the extraction, validation, and persistence tasks.
- **Processing DAG:** This script uses FileSensor as its sole trigger mechanism, which continuously monitors the `datalake_bronze` directory for new JSON files. When it detects one, it automatically triggers the transformation pipeline and saves the result as a Parquet file in `DataLake_Silver`. This design completely decouples ingestion from processing: if the Ingestion DAG fails, the Processing DAG simply does not trigger, without causing cascading errors.

3 Preprocessing Pipeline Definition

This section documents all data cleaning and transformation steps applied to move data from the Bronze layer (raw JSON) to the Silver layer (structured Parquet). The pipeline is divided into two independent branches: one for structured quantitative data from the Last.fm API, and one for unstructured opinion text from Reddit. Each decision is grounded in data quality observations made on the samples collected in Workshop #1.

3.1 Last.fm Silver Pipeline

The Last.fm pipeline processes two JSON sources independently — `lastfm_top_artists` and `lastfm_top_track` — applying identical structural steps to each. Both pipelines are implemented in `lastfm_silver.py`.

3.1.1 Step 1 — Invalid Record Filtering

Before any transformation, records that cannot contribute meaningful data are discarded. A record is considered invalid if:

- The `name` field is empty or null. Artist and track names are the primary key for all downstream aggregations; a nameless record provides no analytical value.
- The `playcount` field equals zero. A zero playcount indicates either a data error or a record that has never been played, which would introduce noise into popularity rankings.

Justification: Inspection of the Bronze samples revealed that a small number of records returned by the API carry empty `mbid` fields and occasionally empty name strings, likely due to entries in transition or API indexing artifacts. Filtering these records before schema enforcement prevents downstream failures.

3.1.2 Step 2 — Text Normalization and Tokenization

All name fields (`name` for artists; `name` and `artist_name` for tracks) are passed through the `build_name_tokens()` pipeline to produce a normalized, noise-free token string. The pipeline applies four sequential functions:

Table 1: Text normalization pipeline — `build_name_tokens()`

Order	Function	Input example	Output example
1	`normalize_text()`	`" The Weeknd "`	`"the weeknd"`
2	`clean_html_entities()`	`"AC&amp;DC"`	`"AC&DC"`
3	`clean_punctuation()`	`"don't @ me!"`	`"don't me"`
4	`remove_links()`	`"artist http://last.fm/x"`	`"artist "`

Justification: The Workshop #1 API samples showed that artist names frequently contain HTML entities (e.g. `&` for bands like AC/DC) and occasional URL fragments in the `bio` fields. Punctuation is preserved selectively (`/`, `&`, `-`, `'`) because these characters carry semantic meaning in music artist names (e.g. *Guns N' Roses*, *AC/DC*).

3.1.3 Step 3 — Deduplication

The `deduplicate()` function applies three prioritized passes to eliminate redundant records within each ingestion batch:

Table 2: Deduplication priority order — Last.fm pipeline

Priority	Condition	Resolution
1	Exact duplicate rows	Keep the first occurrence via `drop_duplicates(keep="first")`.
2	Same name, different `ingested_at`	Keep the most recent record (sort descending by `ingested_at`).
3	Same name, different `playcount`	Keep the record with the highest `playcount`.

For tracks, the deduplication key is the composite `name||artist_name` to correctly distinguish tracks with the same title by different artists (e.g. *Hallelujah* by multiple artists).

Justification: Since the Bronze DAG runs daily and the Last.fm chart rankings change slowly, consecutive ingestion batches are likely to contain overlapping records. Without deduplication, the Silver layer would accumulate redundant rows across Parquet files, inflating record counts in the Gold aggregations.

3.1.4 Step 4 — Schema Enforcement and Type Casting

The `_enforce_schema()` function validates and casts the DataFrame to the target PyArrow schema:

- **Required fields**: validated for null presence; raises `ValueError` if any null is found.
- **Optional fields** (`mbid`, `artist_mbid`): null values are filled with the sentinel value `'unknown'`.
- **Type casting**: `playcount` and `listeners` are cast from string (as returned by the API) to `int64`. Name fields are cast to `string` (PyArrow nullable string type).

Justification: The Last.fm API returns all numeric fields as strings (e.g. `"playcount": "385642100"`). Casting to `int64` is required to enable numeric aggregations in the Gold layer. The `mbid` field was frequently empty in Workshop #1 samples, confirming the need for a sentinel value rather than dropping the field entirely.

3.1.5 Step 5 — Parquet Persistence

The cleaned DataFrame is written to `datalake_silver/` as a Parquet file with Snappy compression via `_save_parquet()`. The filename follows the convention `lastfm_top_{source}_YYYYMMDD_HHMMSS.parquet`.

Justification: Parquet with Snappy compression provides significantly better storage efficiency and query performance compared to JSON or CSV, which is critical when the Gold DAG reads all accumulated Silver files simultaneously using PySpark's folder-read pattern.

3.2 Reddit Silver Pipeline

The Reddit pipeline processes unstructured opinion text from `r/musicsuggestions` posts. It is implemented in `reddit_silver.py` and applies a full NLP cleaning and feature extraction pipeline derived from the exploratory notebook `esqm_limpieza.ipynb`. The pipeline produces one enriched row per comment, with classification and entity extraction features ready for Gold-layer sentiment and storytelling aggregations.

3.2.1 Step 1 — Null Standardization

The function `normalize_nulls()` maps all semantically null values to `NaN` before any downstream processing:

Table 3: Null standardization — `normalize_nulls()`

Input value	Result
`None`	`NaN`
`"null"`	`NaN`
`"none"`	`NaN`
`"nan"`	`NaN`
`""`	`NaN`
`"[deleted]"`	`NaN`
`"[removed]"`	`NaN`

Justification: Reddit posts frequently contain deleted or removed comments that appear as literal strings `[deleted]` or `[removed]`. The Workshop #1 scraping sample confirmed this pattern. These strings would pass standard null checks and introduce noise into NLP features if not standardized early.

3.2.2 Step 2 — Record Filtering and Score Imputation

Posts without a `title` or without a `comments` list are discarded, as they provide no textual content for analysis. The `score` field is imputed with `0` when null or non-numeric, using `pd.to_numeric(errors="coerce")`.

Justification: Score represents the community relevance signal of a post. Dropping posts with missing scores would reduce the dataset unnecessarily; imputing with 0 is conservative and preserves the text content while marking the post as unranked.

3.2.3 Step 3 — Comment Explosion

Each post may contain multiple comments stored as a list. The pipeline applies `df.explode("raw_comment")` to produce one row per comment. Empty strings and non-string entries are filtered out after explosion.

Justification: Sentiment analysis operates at the comment level, not the post level. Exploding to one row per comment is the correct granularity for downstream NLP processing and enables per-comment classification.

3.2.4 Step 4 — Multi-sentence Comment Splitting

The function `split_multiple_comments()` further splits comments thatcontain multiple sentences or sub-recommendations, using line breaks and uppercase-after-parenthesis as split signals:

```
parts = re.split(r'\n+|(?<=\))\s+(?=[A-Z])', content)
```

Justification: Reddit users in `r/musicsuggestions` frequently list multiple song recommendations in a single comment block separated by newlines. Splitting these into individual records improves the precision of the music entity extraction step (Step 9).

3.2.5 Step 5 — Text Cleaning Pipeline

Each comment is cleaned by applying four functions sequentially to produce `clean_comment`:

Table 4: NLP text cleaning pipeline — Reddit comments

Order	Function	Input example	Output example
1	`clean_html_entities()`	`"AC&amp;DC is great"`	`"AC&DC is great"`
2	`remove_links()`	`"listen at http://spotify.com"`	`"listen at "`
3	`clean_punctuation()`	`"love it!!!  #music"`	`"love it music"`
4	`normalize_text()`	`" Love IT "`	`"love it"`

Justification: Reddit comments contain a high density of URLs (Spotify links, YouTube links), HTML entities from the API response, and heavy punctuation (exclamation marks, hashtags). These elements do not contribute to semantic analysis and would inflate token counts. Links in particular were observed in the Workshop #1 scraping sample as a dominant source of noise.

3.2.6 Step 6 — Tokenization

The `tokenize()` function splits the `clean_comment` by whitespace. The resulting token list is serialized as a string for Parquet compatibility:

```
df["tokens"] = df["clean_comment"].apply(lambda c: str(tokenize(c)))
```

Justification: Parquet does not natively support list-of-string columns without using nested types, which would complicate PySpark reads in the Gold layer. Serializing as a string preserves the token information while maintaining schema simplicity.

3.2.7 Step 7 — Comment Classification

The function `classify_comment()` assigns each comment a `comment_type` and a `confidence` score based on structural and lexical patterns:

Table 5: Comment classification types and detection logic

Type	Detection logic
`recommendation`	Comment matches a music pattern (dash, by, colon) and does not contain opinion contrast markers.
`opinion`	Comment contains contrast markers (*but, however, although, though*) or is longer than 6 words, with no music pattern detected.
`mixed`	Comment matches a music pattern AND contains opinion markers.
`other`	Comment matches no pattern and is too short to classify as an opinion.

Three structural patterns are detected to identify music recommendations:

Table 6: Music recommendation patterns detected by `classify_comment()`

Pattern	Regex signal	Example
`dash`	`Artist - Song`	*The Beatles - Hey Jude*
`by`	`Song by Artist`	*Bohemian Rhapsody by Queen*
`colon`	`Artist: Song`	*Radiohead: Creep*

The `confidence` score is computed additively based on: pattern presence (+0.45), word count range (+0.25 for $\leq$5 words, +0.15 for $\leq$10), penalized by contrast presence (-0.20), mixed type (-0.10), excessive punctuation (-0.10), and very short comments (-0.20). The score is clamped to $[0.0, 1.0]$.

Justification: The Workshop #1 Reddit sample revealed that textttr/musicsuggestions posts are dominated by short recommendation-style comments following these exact three patterns. Classifying comments by type enables the Gold layer to filter or weight records by their analytical value for the storytelling dashboard.

3.2.8 Step 8 — Artist and Song Extraction

The function `extract_artist_song()` extracts structured `artist` and `song` fields from comments classified with a known `pattern_type`. Each pattern applies a pattern-specific extraction rule with a confidence scoring mechanism:

Table 7: Extraction logic per pattern type

Pattern	Artist	Song	Confidence threshold
`dash`	Left of `-`	Right of `-`	≥ 0.7 to emit result
`by`	After `by`	Before `by`	Always emitted
`colon`	Left of `:`	Right of `:`	Always emitted

Justification: Extracting artist and song names from free text is the primary analytical goal of the Reddit pipeline. These structured fields enable the Gold layer to aggregate community sentiment by artist and song, directly supporting the user story: *"As a Music Label Analyst, I want to compare listener statistics from Last.fm with sentiment volume on social platforms"*.

3.2.9 Step 9 — Outlier Capping (IQR)

Two numeric fields are capped using the IQR method to reduce the influence of extreme values in Gold-layer aggregations:

Table 8: Outlier capping via IQR method

Field	Capped field	Rationale
`score`	`score_capped`	Reddit post scores are highly skewed; viral posts can reach thousands while most remain below 50. Capping prevents a single viral post from dominating weighted aggregations.
`word_count`	`word_count_capped`	A small number of comments are extremely long (copied lyrics, full articles). Capping prevents these from skewing mean word count statistics in the governance KPIs.

The capping formula is: $\text{clip}(x,\ Q_1 - 1.5 \cdot \text{IQR},\ Q_3 + 1.5 \cdot \text{IQR})$, applied via `pd.Series.clip()`.

Justification: The Workshop #1 scraping sample showed high variance in post scores, with most posts scoring below 10 and occasional outliers above 500. The original values are preserved in `score` and `word_count`; the capped versions provide robust alternatives for use in visualizations.

3.2.10 Step 10 — Deduplication

Exact duplicates are removed using the composite key `(post_id, clean_comment)`:

```
df = df.drop_duplicates(subset=["post_id", "clean_comment"])
```

Justification: The comment splitting step (Step 4) and the explosion step (Step 3) can occasionally produce duplicate `clean_comment` values for the same post if a user writes the same recommendation twice or if the splitting regex produces identical sub-strings. Deduplication on the composite key preserves distinct comments from the same post while removing true duplicates.

3.2.11 Step 11 — Noise Removal

After all transformation steps, rows where `clean_comment` is empty or null are removed. These arise when a comment consisted entirely of URLs, punctuation, or HTML entities that were stripped in the cleaning step.

Justification: An empty `clean_comment` after the full cleaning pipeline means the comment carried no textual content useful for NLP. Keeping these rows would produce zero-length token lists and `other` classifications that add no analytical value.

3.2.12 Step 12 — Schema Enforcement and Type Casting

Identical to the Last.fm pipeline, `_enforce_schema()` validates required fields, fills optional fields (`pattern_type`, `artist`, `song`) with `'unknown'`, and casts all columns to the types defined in

`REDDIT_SCHEMA`. Boolean fields (`has_music_pattern`, `has_contrast`) are explicitly cast to `bool` to avoid Parquet serialization issues with mixed types.

3.3 Pipeline Comparison Summary

Table 9: Preprocessing pipeline comparison — Last.fm vs Reddit

Aspect	**Last.fm Pipeline**	**Reddit Pipeline**
Input format	Structured JSON (list of artist/-track objects)	Semi-structured JSON (list of posts with comment lists)
Granularity	One row per artist / one row per track	One row per comment (after explode and split)
Text fields processed	`name`, `artist_name`	`title`, `raw_comment`, `clean_comment`
NLP features generated	`name_tokens`, `artist_name_tokens`	`tokens`, `comment_type`, `confidence`, `has_music_pattern`, `artist`, `song`
Outlier handling	Not required (API returns clean numeric data)	IQR capping on `score` and `word_count`
Deduplication key	`name` (artists) / `name||artist_name` (tracks)	`(post_id, clean_comment)`
Output format	Parquet, Snappy, strict schema	Parquet, Snappy, strict schema
Primary Gold use	Governance KPIs, popularity metrics	Storytelling aggregations, sentiment analysis

4 Docker Stack Description

Docker is a containerization platform that enables developers to package applications and their dependencies into isolated environments called containers. These containers ensure consistency across different stages of development, testing, and production, eliminating issues related to environment discrepancies. By using Docker, applications can run reliably regardless of the underlying infrastructure.

In this project, Docker is used to orchestrate multiple services required for a data engineering workflow. Instead of manually installing and configuring each component, Docker Compose is employed to define and manage all services in a declarative manner. This approach simplifies deployment, as the entire system can be started with a single command, ensuring that all dependencies are properly configured and connected.

The use of Docker also improves scalability and maintainability. Services can be easily modified, replaced, or extended without affecting the rest of the system. Additionally, containerization promotes modular design, allowing each component, such as the database or workflow engine, to operate independently while still being part of an integrated architecture.

From an operational perspective, Docker provides essential features such as volume management for data persistence, networking for inter-service communication, and environment variable injection for flexible configuration. These capabilities make it a suitable choice for building robust and reproducible data platforms.

5 Docker Stack Architecture

The system architecture is composed of several interconnected services that work together to provide a complete data orchestration platform based on Apache Airflow and PostgreSQL.

5.1 Core Services

- **PostgreSQL:** This service acts as the central data storage component. It hosts both the Airflow metadata database and an additional database dedicated to the EarthShake application. The secondary database is automatically created during container initialization through an SQL script. Data persistence is ensured through the use of Docker volumes.
- **Airflow Initialization:** This service runs only once during startup and is responsible for preparing the Airflow environment. It performs database migrations and creates an administrative user, ensuring that the system is correctly configured before other services begin execution.
- **Airflow Webserver:** This component provides a graphical user interface that allows users to visualize DAGs, monitor executions, and manage workflows. It depends on both the PostgreSQL service and the successful completion of the initialization process.
- **Airflow Scheduler:** This service is responsible for scheduling and executing workflows. It continuously monitors DAGs and triggers tasks based on their defined schedules, enabling automated data pipeline execution.

To ensure consistency across Airflow services, a shared configuration block is defined. This configuration includes common environment variables, database connection strings, and mounted volumes, reducing redundancy and simplifying maintenance.

5.2 Environment Variables

The configuration of the system relies on environment variables defined in an external `.env` file. This approach enhances flexibility and security by separating sensitive information from the application code.

- **POSTGRES_USER:** Username for the PostgreSQL database.
- **POSTGRES_PASSWORD:** Password for database authentication.
- **POSTGRES_HOST:** Hostname of the PostgreSQL service.
- **POSTGRES_PORT:** Port used by PostgreSQL.
- **AIRFLOW_DB:** Database name for Airflow metadata.
- **ES_DB:** Database name for the EarthShake application.
- **AIRFLOW_ADMIN_USER:** Administrator username for Airflow.
- **AIRFLOW_ADMIN_PASSWORD:** Administrator password.

- **AIRFLOW_ADMIN_EMAIL:** Administrator email address.

These variables are injected at runtime and are used to configure database connections, authentication, and service behavior dynamically.

5.3 Volumes

Docker volumes are used to ensure data persistence and enable data sharing between containers. This prevents data loss when containers are stopped or recreated and allows services to access shared resources consistently.

- **Postgres_data:** Stores PostgreSQL data files, ensuring persistence across container restarts.
- **DAGs:** Contains Airflow DAG definitions, allowing workflows to be updated without rebuilding containers.
- **SQL:** Stores SQL scripts used for initialization and database management.
- **Datalake:** Used to store raw and processed data for ETL workflows.

The use of volumes ensures a reliable and maintainable data management strategy within the Dockerized environment.

6 DAG Documentation

6.1 dag_lastfm_ingest

The `lastfm_ingest` DAG is the Bronze layer ingestion component of the *Music Artists & Albums Public Perception* pipeline. Its responsibility is to extract daily global rankings from the Last.fm API through two separate endpoints and persist each response as a raw JSON file inside dedicated subfolders within `datalake_bronze/`. pon successful completion, the DAG automatically triggers the Silver processing DAG (`lastfm_silver`) via a `TriggerDagRunOperator`, initiating the transformation and preprocessing flow without blocking the Bronze execution.

Each DAG run produces two JSON files, one per data source, stored in separate subfolders to maintain source isolation and facilitate FileSensor targeting in the Silver DAG:

```
datalake_bronze/
|-- lastfm_top_tracks/
|   `-- lastfm_top_tracks_YYYYMMDD_HHMMSS.json
`-- lastfm_top_artists/
    `-- lastfm_top_artists_YYYYMMDD_HHMMSS.json
```

The datetime-based filename convention guarantees idempotency: multiple runs on the same day produce distinct files without overwriting previous ingestion outputs.

6.1.1 Task Description

Table 10: Task descriptions for `lastfm_ingest`

Task	Type	Description
`extract_top_tracks`	`@task`	Calls `chart.getTopTracks`. Saves raw JSON in `lastfm_top_tracks/`. Returns the file path.
`extract_top_artists`	`@task`	Calls `chart.getTopArtists`. Saves raw JSON in `lastfm_top_artists/`. Returns the file path.
`validate_bronze_files`	`@task`	Validates existence and integrity of both JSON files. Returns a consolidated ingestion summary.
`trigger_silver_dag`	`TriggerDagRunOperator`	Triggers `lastfm_silver` without blocking Bronze (`wait_for_completion=False`).

6.1.2 Task: `extract_top_tracks`

Calls `chart.getTopTracks` and persists the response in `datalake_bronze/lastfm_top_tracks/`.

6.1.3 Relevant Fields per Track

Table 11: Fields extracted from `chart.getTopTracks`

Field	API Type	Description
`name`	string	Track name.
`playcount`	string	Total global play count.
`listeners`	string	Unique listener count.
`mbid`	string	MusicBrainz unique identifier.
`url`	string	Track URL on Last.fm.
`artist.name`	string	Artist name.
`artist.mbid`	string	Artist MusicBrainz ID.
`artist.url`	string	Artist URL on Last.fm.

6.1.4 Persisted JSON Structure

```
{
    "_metadata": {
        "ingested_at": "2026-05-28T23:28:36.595072",
        "source": "last.fm",
        "method": "chart.getTopTracks",
        "track_count": 50
    },
    "data": {
        "tracks": {
            "track": [
                {
                    "name": "tHe CuRe",
                    "duration": "297",
                    "playcount": "2769035",
                    "listeners": "313129",
                    "mbid": "1de149c9-1f2d-491e-b628-6dde34e13b62",
                    "url": "https://www.last.fm/music/Olivia+Rodrigo/_/tHe+CuRe",
                    "streamable": {
                        "#text": "0",
                        "fulltrack": "0"
                    },
                    "artist": {
                        "name": "Olivia Rodrigo",
                        "mbid": "6925db17-f35e-42f3-a4eb-84ee6bf5d4b0",
                        "url": "https://www.last.fm/music/Olivia+Rodrigo"
                    },
```


Figure 2: Json data from the track extraction

6.1.5 Task: `extract_top_artists`

Calls `chart.getTopArtists` and persists the response in `datalake_bronze/lastfm_top_artists/`.

6.1.6 Relevant Fields per Artist

Table 12: Fields extracted from `chart.getTopArtists`

Field	API Type	Description
`name`	string	Artist name.
`playcount`	string	Total global play count.
`listeners`	string	Unique listener count.
`mbid`	string	MusicBrainz unique identifier.
`url`	string	Artist URL on Last.fm.

6.1.7 Persisted JSON Structure

```
{
    "_metadata": {
        "ingested_at": "2026-05-28T23:28:36.415540",
        "source": "last.fm",
        "method": "chart.getTopArtists",
        "artist_count": 50
    },
    "data": {
        "artists": {
            "artist": [
                {
                    "name": "Drake",
                    "playcount": "1208415234",
                    "listeners": "6784020",
                    "mbid": "9fff2f8a-21e6-47de-a2b8-7f449929d43f",
                    "url": "https://www.last.fm/music/Drake",
                    "streamable": "0",
                    "image": [
                        {
                            "#text": "https://lastfm.freetls.fastly.net/i/u/34s/2a96cbd8b46e442fc41c2b86b821562f.png",
                            "size": "small"
                        },
```


Figure 3: JSON data from the top_artist extraction

6.2 Github Actions

To keep things running automatically without relying on a local machine, the project uses a GitHub Actions workflow defined in lastfm_ingestion.yml. This file is basically in charge of running the ingest_lastfm.py script every day at 12:00 PM Colombia Time (COT), which is handled by a cron expression set to 0 17 * * * in UTC. On top of that, the workflow can also be triggered manually straight from the GitHub Actions tab whenever needed, which comes in handy for testing or re-running a specific day. Each time it fires, GitHub spins up a fresh Ubuntu runner, sets up Python 3.11, installs requests, and runs the ingestion script with the LASTFM_API_KEY pulled securely from the repository secrets, so no credentials ever touch the codebase directly. Once the script finishes, the workflow checks whether any new files were written to datalake_bronze/ and, if so, commits and pushes them back to the repo under a bot account with a date-stamped message. The smart part here is that it only commits when there's actually something new, avoiding pointless empty commits. This essentially turns the GitHub repo into a lightweight but reliable versioned Bronze layer, where every daily run leaves a clean, traceable record of the raw data pulled from Last.fm.

6.3 dag_lastfm_silver

The `lastfm_silver` DAG is the Silver layer processing component of the *Music Artists & Albums Public Perception* pipeline. Its responsibility is to detect new raw JSON files in `datalake_bronze/`, apply a full normalization and cleaning pipeline, and persist the results as structured Parquet files inside dedicated subfolders within `datalake_silver/`.

The DAG is triggered automatically by the Bronze DAG via `TriggerDagRunOperator` upon successful ingestion, ensuring that Silver processing always operates on the most recently available Bronze data.

Each DAG run produces two Parquet files — one for top artists and one for top tracks —following the same subfolder isolation pattern as the Bronze layer:

```
datalake_silver/
|-- lastfm_top_artists/
|   `-- lastfm_top_artists_YYYYMMDD_HHMMSS.parquet
`-- lastfm_top_tracks/
    `-- lastfm_top_tracks_YYYYMMDD_HHMMSS.parquet
```

6.3.1 Target Schemas

Each Silver Parquet file enforces a strict schema defined via PyArrow. Fields marked as required are validated for nulls before persistence; optional fields are filled with `'unknown'` when absent.

Table 13: Silver schema — `lastfm_top_artists`

Field	Type	Required	Default	Description
`name`	string	Yes	–	Artist name as returned by the API.
`name_tokens`	string	Yes	–	Cleaned and tokenized version of the name.
`playcount`	int64	Yes	–	Total global play count.
`listeners`	int64	Yes	–	Unique listener count.
`mbid`	string	No	unknown	MusicBrainz unique identifier.
`ingested_at`	string	Yes	–	ISO timestamp from Bronze metadata.

Table 14: Silver schema — `lastfm_top_tracks`

Field	Type	Required	Default	Description
`name`	string	Yes	–	Track name as returned by the API.
`name_tokens`	string	Yes	–	Cleaned and tokenized version of the track name.
`duration_sec`	int64	Yes	–	Track duration in seconds.
`playcount`	int64	Yes	–	Total global play count.
`listeners`	int64	Yes	–	Unique listener count.
`mbid`	string	No	unknown	Track MusicBrainz identifier.
`artist_name`	string	Yes	–	Artist name.
`artist_name_tokens`	string	Yes	–	Cleaned and tokenized version of the artist name.
`artist_mbid`	string	No	unknown	Artist MusicBrainz identifier.
`ingested_at`	string	Yes	–	ISO timestamp from Bronze metadata.

6.3.2 Text Cleaning Pipeline

All `name_tokens` and `artist_name_tokens` fields are produced by the function `build_name_tokens()`, which applies four sequential cleaning steps:

Table 15: Text normalization pipeline applied to name fields

Step	Function	Description
1	`normalize_text()`	Strip whitespace, lowercase, collapse multiple spaces.
2	`clean_html_entities()`	Decode HTML entities (e.g. `&` → `&`, `&#39;` → `'`).
3	`clean_punctuation()`	Remove punctuation except `/`, `&`, `-`, `'` (relevant in artist names).
4	`remove_links()`	Remove URLs and Markdown-format links.

6.3.3 Deduplication Strategy

The function `deduplicate()` applies three prioritized passes to remove redundant records within each ingestion batch:

Table 16: Deduplication priority order

Priority	Condition	Resolution
1	Exact duplicate rows	Keep the first occurrence.
2	Same name, different ingestion timestamp	Keep the most recent (`ingested_at` descending).
3	Same name, different playcount	Keep the record with the highest `playcount`.

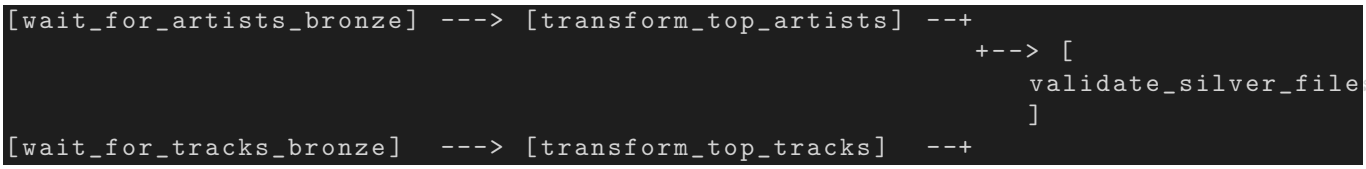
For tracks, the deduplication key is a composite of `name` and `artist_name` (joined as `name||artist_name`) to correctly distinguish tracks with identical titles by different artists.

6.3.4 Task Description

Table 17: Task descriptions for `lastfm_silver`

Task	Type	Description
`wait_for_artists_bronze`	`FileSensor`	Polls `datalake_bronze/lastfm_top_artists/` every 30 s (timeout 10 min) until at least one JSON file is detected.
`wait_for_tracks_bronze`	`FileSensor`	Polls `datalake_bronze/lastfm_top_tracks/` every 30 s (timeout 10 min) until at least one JSON file is detected.
`transform_top_artists`	`@task`	Loads the latest Bronze JSON for artists, applies the full cleaning and deduplication pipeline, enforces the schema, and saves the result as Parquet in `datalake_silver/lastfm_top_artists/`.
`transform_top_tracks`	`@task`	Loads the latest Bronze JSON for tracks, applies the full cleaning and deduplication pipeline, enforces the schema, and saves the result as Parquet in `datalake_silver/lastfm_top_tracks/`.
`validate_silver_files`	`@task`	Validates that both Parquet files exist, match the defined schema, and contain at least one row. Returns a consolidated summary.

6.3.5 Task Flow Diagram



The two FileSensors and the two transformation tasks run in parallel. The validation task waits for both transformation outputs before executing.

6.3.6 Task: `transform_top_artists`

Reads the most recent Bronze JSON from `lastfm_top_artists/`, builds a normalized DataFrame, and persists it as a Parquet file in `datalake_silver/lastfm_top_artists/`.

Processing Steps

1. Load the latest JSON file using `_get_latest_bronze_file()`.
2. Extract the `artists.artist` list and the `ingested_at` timestamp from `_metadata`.
3. Build a row per artist, computing `name_tokens` via `build_name_tokens()`.
4. Filter out invalid records: empty `name` or `playcount = 0`.
5. Deduplicate using `deduplicate(key="name")`.
6. Enforce schema and cast types via `_enforce_schema()`.
7. Persist as Parquet using `_save_parquet()` with Snappy compression.

Silver Output Sample

```
df_sil_rute = '../datalake_silver/lastfm_top_artists/lastfm_top_artists_20260529_014902.parquet'
df_paq = pd.read_parquet(df_sil_rute)
df_paq.head(30)
```

✓ 0.0s

	name	name_tokens	playcount	listeners	mbid	ingested_at
0	BTS	bts	4109993813	2487399	0d79fe8e-ba27-4859-bb8c-2f255f346853	2026-05-29T01:48:55.869779
1	Taylor Swift	taylor swift	3711214920	5982694	20244d07-534f-4eff-b4d4-930878889970	2026-05-29T01:48:55.869779
2	Lana Del Rey	lana del rey	1598202820	5302693	b7539c32-53e7-4908-bda3-81449c367da6	2026-05-29T01:48:55.869779
3	Kanye West	kanye west	1561865290	7995538	unknown	2026-05-29T01:48:55.869779
4	Radiohead	radiohead	1389546781	8287544	a74b1b7f-71a5-4011-9441-d0b5e4122711	2026-05-29T01:48:55.869779
5	Drake	drake	1208415234	6784020	9fff2f8a-21e6-47de-a2b8-7f449929d43f	2026-05-29T01:48:55.869779
6	The Weeknd	the weeknd	1138436080	5366235	c8b03190-306c-4120-bb0b-6f2ebfc06ea9	2026-05-29T01:48:55.869779
7	Ariana Grande	ariana grande	1132554717	4527837	f4fdbb4c-e4b7-47a0-b83b-d91bbfcfa387	2026-05-29T01:48:55.869779
8	The Beatles	the beatles	1112185648	6552684	b10bbbfc-cf9e-42e0-be17-e2c3e1d2600d	2026-05-29T01:48:55.869779
9	Tyler, The Creator	tyler the creator	1064492915	4371141	f6beac20-5dfe-4d1f-ae02-0b0a740aafd6	2026-05-29T01:48:55.869779
10	Lady Gaga	lady gaga	1045518366	7772676	650e7db6-b795-4eb5-a702-5ea2fc46c848	2026-05-29T01:48:55.869779
11	Kendrick Lamar	kendrick lamar	1042634849	5140481	381086ea-f511-4aba-bdf9-71c753dc5077	2026-05-29T01:48:55.869779
12	Arctic Monkeys	arctic monkeys	944116353	7151088	ada7a83c-e3e1-40f1-93f9-3e73dbc9298a	2026-05-29T01:48:55.869779
13	Billie Eilish	billie eilish	848236311	4273090	f4abc0b5-3f7a-4eff-8f78-ac078dbce533	2026-05-29T01:48:55.869779
14	Frank Ocean	frank ocean	786563135	4302551	e520459c-dff4-491d-a6e4-c97be35e0044	2026-05-29T01:48:55.869779
15	Coldplay	coldplay	785537565	9155733	cc197bad-dc9c-440d-a5b5-d52ba2e14234	2026-05-29T01:48:55.869779
16	Linkin Park	linkin park	754904756	6982027	f59c5520-5f46-4d2c-b2c4-822eabf53419	2026-05-29T01:48:55.869779
17	Charli xcx	charli xcx	734132350	3814502	260b6184-8828-48eb-945c-bc4cb6fc34ca	2026-05-29T01:48:55.869779
18	Beyoncé	beyoncé	706936483	6427066	859d0860-d480-4efd-970c-c05d5f1776b8	2026-05-29T01:48:55.869779
19	Travis Scott	travis scott	690776741	3369939	e4a51f17-a57b-47b1-b37b-f552d0f8e9e6	2026-05-29T01:48:55.869779

Figure 4: Sample Parquet output from `transform_top_artists`

6.3.7 Task: `transform_top_tracks`

Reads the most recent Bronze JSON from `lastfm_top_tracks/`, builds a normalized DataFrame, and persists it as a Parquet file in `datalake_silver/lastfm_top_tracks/`.

Processing Steps

1. Load the latest JSON file using `_get_latest_bronze_file()`.
2. Extract the `tracks.track` list and the `ingested_at` timestamp from `_metadata`.
3. Build a row per track, computing `name_tokens` and `artist_name_tokens` via `build_name_tokens()`.
4. Filter out invalid records: empty `name`, empty `artist_name`, or `playcount = 0`.
5. Deduplicate using a composite key `name||artist_name` to handle tracks with identical titles by different artists.
6. Enforce schema and cast types via `_enforce_schema()`.
7. Persist as Parquet using `_save_parquet()` with Snappy compression.

Silver Output Sample

```
df_sil_rute = '../datalake_silver/lastfm_top_tracks/lastfm_top_tracks_20260529_014902.parquet'
df_paq = pd.read_parquet(df_sil_rute)
df_paq.head(30)
```

✓ 0.0s Python

	name	name_tokens	duration_sec	playcount	listeners	mbid	artist_name	artist_name_tokens	artist_mbid	ingested_at
0	SWIM	swim	159	198661735	456991	6f33dc05-cdc0-4a2f-8039-e8fed082eec6	BTS	bts	0d79fe8e-ba27-4859-bb8c-2f255f346853	2026-05-29T01:48:55.860499
1	Creep	creep	235	61244353	4114888	012e70cd-4ef6-37bf-8ebf-02d318fe5151	Radiohead	radiohead	a74b1b7f-71a5-4011-9441-d0b5e4122711	2026-05-29T01:48:55.860499
2	Good Luck, Babe!	good luck babe	218	60643840	2100652	1469986c-a292-4d2c-a253-89ed5dc56bf2	Chappell Roan	chappell roan	56a55378-f155-48de-80a5-d80104221267	2026-05-29T01:48:55.860499
3	Pink + White	pink white	290	59944740	2462316	0e360068-12c7-4012-853c-6a6011e0b36a	Frank Ocean	frank ocean	e520459c-dff4-491d-a6e4-c97be35e0044	2026-05-29T01:48:55.860499
4	505	505	305	58803216	3050877	00d0985d-278e-4a72-b959-fc184a1ce990	Arctic Monkeys	arctic monkeys	ada7a83c-e3e1-40f1-93f9-3e73dbc9298a	2026-05-29T01:48:55.860499
5	See You Again (feat. Kali Uchis)	see you again feat kali uchis	180	58283364	2582928	681a3318-d896-4696-8745-0b9aaf642040	Tyler, The Creator	tyler the creator	f6beac20-5dfe-4d1f-ae02-0b0a740aafd6	2026-05-29T01:48:55.860499
6	Body to Body	body to body	189	56037565	403722	6dfdfc61-89ff-451a-a5e9-5319065beae7	BTS	bts	0d79fe8e-ba27-4859-bb8c-2f255f346853	2026-05-29T01:48:55.860499
7	Hooligan	hooligan	182	53925751	379449	a045f252-ea1a-42c3-b7a5-f69979e39394	BTS	bts	0d79fe8e-ba27-4859-bb8c-2f255f346853	2026-05-29T01:48:55.860499
8	2.0	20	169	53377250	349925	1d6222e0-77eb-4047-b4fb-045a0c5636d3	BTS	bts	0d79fe8e-ba27-4859-bb8c-2f255f346853	2026-05-29T01:48:55.860499
9	Mr. Brightside	mr brightside	222	51873262	3711285	0157fd48-a126-4af1-ad1f-4fbb37911b90	The Killers	the killers	95e1ead9-4d31-4808-a7ac-32c3614c116b	2026-05-29T01:48:55.860499
10	FYA	fya	180	51566263	355845	802e86e2-5da4-40c5-a265-933eb64d864b	BTS	bts	0d79fe8e-ba27-4859-bb8c-2f255f346853	2026-05-29T01:48:55.860499

Figure 5: Sample Parquet output from `transform_top_tracks`

6.3.8 Task: `validate_silver_files`

Verifies the integrity of both Parquet files produced in the same DAG run. Raises an exception and fails the DAG if any validation check does not pass.

Validations Performed

- File existence check via `os.path.exists()`.
- Non-empty row count (`table.num_rows > 0`).
- Schema field presence: all expected column names must exist in the Parquet schema.
- Null count report per file for observability.

Returned Summary Structure


```
{
    "artists": {
        "filepath":   ".../lastfm_top_artists_20260528_103000.parquet",
        "rows":       50,
        "columns":    ["name", "name_tokens", "playcount", "listeners",
                       "mbid", "ingested_at"],
        "top_name":   "The Weeknd",
        "null_count": 0
    },
    "tracks": {
        "filepath":   ".../lastfm_top_tracks_20260528_103000.parquet",
        "rows":       50,
        "columns":    ["name", "name_tokens", "duration_sec", "playcount",
                       "listeners", "mbid", "artist_name",
                       "artist_name_tokens", "artist_mbid", "ingested_at"],
        "top_name":   "Blinding Lights",
        "null_count": 0
    }
}
```


6.3.9 Helper Functions

Table 18: Module-level helper functions in `lastfm_silver.py`

Function	Description
`_get_latest_bronze_file(folder)`	Returns the most recent JSON file path in the given Bronze subfolder using alphabetical sort (datetime-based filenames are lexicographically ordered). Raises `FileNotFoundError` if no files are found.
`_enforce_schema(df, schema)`	Validates required fields for nulls, fills optional fields with `'unknown'`, and casts all columns to the types defined in the PyArrow schema.
`_save_parquet(df, schema, folder, filename)`	Converts the DataFrame to a PyArrow Table with the given schema and writes it as a Parquet file with Snappy compression.

6.3.10 Error Handling

Table 19: Error handling in `lastfm_silver`

Condition	Exception	Behavior
No JSON files in Bronze folder	`FileNotFoundError`	`_get_latest_bronze_file()` raises immediately with the folder path.
Required field has null values	`ValueError`	`_enforce_schema()` reports the field name and null count.
Required field missing from DataFrame	`ValueError`	`_enforce_schema()` lists all missing required field names.
Silver Parquet not found after write	`FileNotFoundError`	`validate_silver_files` fails with the missing file path.
Empty Parquet file	`ValueError`	`validate_silver_files` fails indicating the affected source.

6.3.11 DAG Parameters

Table 20: DAG configuration parameters for `lastfm_silver`

Parameter	Value
`dag_id`	`lastfm_silver`
`schedule`	`@daily`
`start_date`	`2024-01-01`
`catchup`	`False`
`retries`	2
`retry_delay`	3 minutes
`tags`	`lastfm, silver, transform`

6.4 Python Dependencies

Table 21: Python dependencies for `lastfm_silver`

Library	Usage
`apache-airflow >= 2.7`	DAG orchestration, `FileSensor`, `@task` decorator.
`pandas >= 1.5`	DataFrame construction, filtering, deduplication, and type casting.
`pyarrow >= 12.0`	Schema definition, Parquet write with Snappy compression.

6.5 dag_reddit_silver

7 Results Evidence

7.1 Github/Ingest

In the next figure we can see the history about the ingestion of data in the repository, which each day there is a push from the API to collect data for our analysis

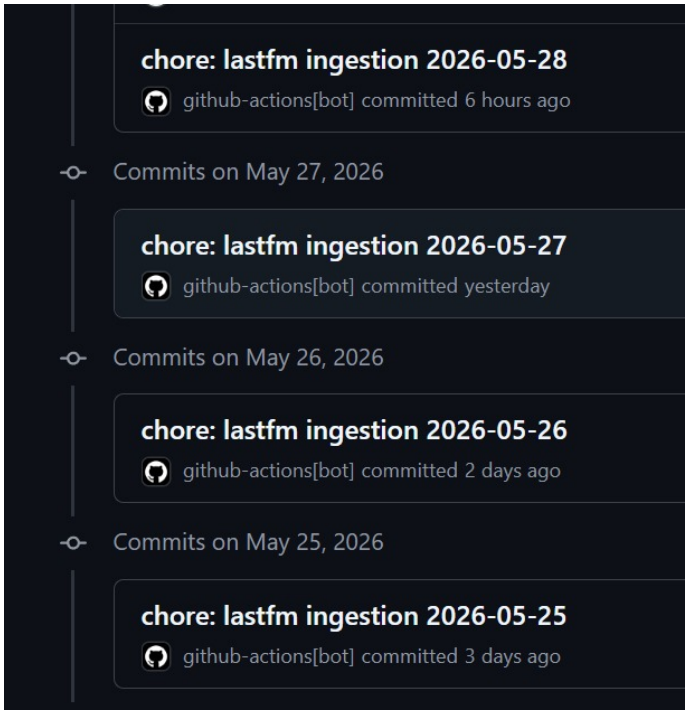


Figure 6: Github Actions

7.2 Airflow DAG Runs

Both DAGs, `lastfm_ingest` and `lastfm_silver`, ran successfully across multiple daily executions without manual intervention. The Airflow UI confirmed that all tasks completed with a success status, and the `TriggerDagRunOperator` correctly chained the Bronze and Silver runs in sequence.

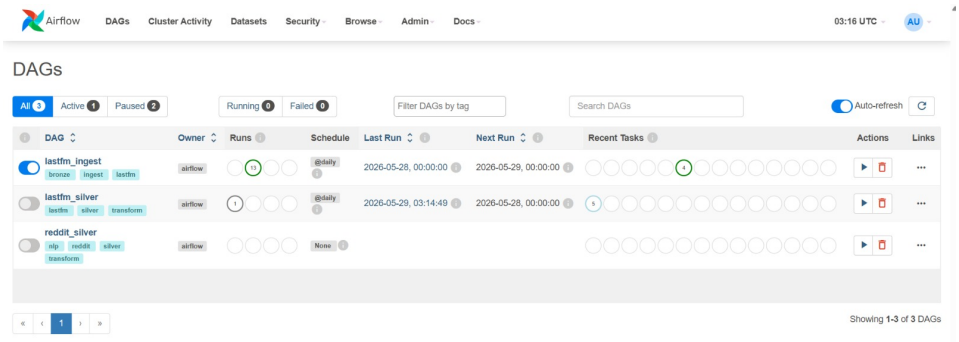


Figure 7: Airflow DAG grid view showing successful daily runs

7.3 Bronze Layer Output

Each ingestion run produced two timestamped JSON files inside their respective subfolders in `datalake_bronze/`, one for top artists and one for top tracks. The file naming convention worked as expected, with no overwrites detected across multiple runs.

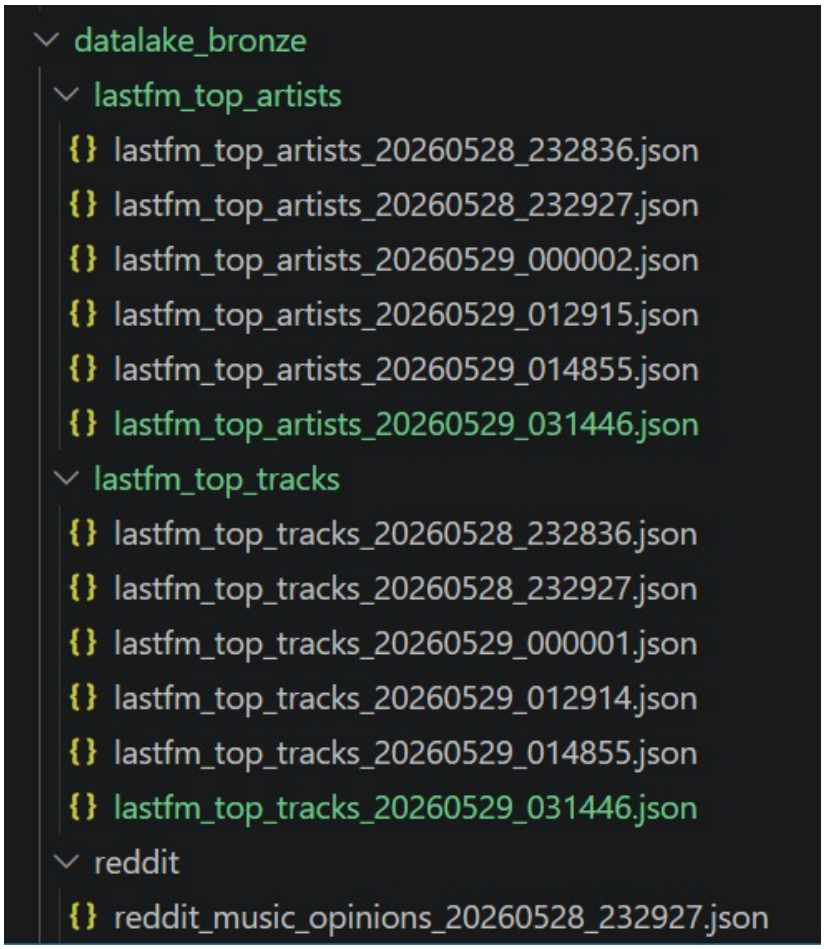


Figure 8: Bronze layer folder showing timestamped JSON files per source

7.4 Silver Layer Output

After each successful Bronze run, the Silver DAG picked up the new files and produced clean, structured Parquet files in `datalake_silver/`. The validation task confirmed that both files passed all integrity checks, with no null values in required fields and record counts matching the expected API response size.

```
df_sil_rute = '../datalake_silver/lastfm_top_artists/lastfm_top_artists_20260529_014902.parquet'
df_paq = pd.read_parquet(df_sil_rute)
df_paq.head(30)
```

✓ 0.0s

	name	name_tokens	playcount	listeners	mbid	ingested_at
0	BTS	bts	4109993813	2487399	0d79fe8e-ba27-4859-bb8c-2f255f346853	2026-05-29T01:48:55.869779
1	Taylor Swift	taylor swift	3711214920	5982694	20244d07-534f-4eff-b4d4-930878889970	2026-05-29T01:48:55.869779
2	Lana Del Rey	lana del rey	1598202820	5302693	b7539c32-53e7-4908-bda3-81449c367da6	2026-05-29T01:48:55.869779
3	Kanye West	kanye west	1561865290	7995538	unknown	2026-05-29T01:48:55.869779
4	Radiohead	radiohead	1389546781	8287544	a74b1b7f-71a5-4011-9441-d0b5e4122711	2026-05-29T01:48:55.869779
5	Drake	drake	1208415234	6784020	9fff2f8a-21e6-47de-a2b8-7f449929d43f	2026-05-29T01:48:55.869779
6	The Weeknd	the weeknd	1138436080	5366235	c8b03190-306c-4120-bb0b-6f2ebfc06ea9	2026-05-29T01:48:55.869779
7	Ariana Grande	ariana grande	1132554717	4527837	f4fdbb4c-e4b7-47a0-b83b-d91bbfcfa387	2026-05-29T01:48:55.869779
8	The Beatles	the beatles	1112185648	6552684	b10bbbfc-cf9e-42e0-be17-e2c3e1d2600d	2026-05-29T01:48:55.869779
9	Tyler, The Creator	tyler the creator	1064492915	4371141	f6beac20-5dfe-4d1f-ae02-0b0a740aafd6	2026-05-29T01:48:55.869779
10	Lady Gaga	lady gaga	1045518366	7772676	650e7db6-b795-4eb5-a702-5ea2fc46c848	2026-05-29T01:48:55.869779
11	Kendrick Lamar	kendrick lamar	1042634849	5140481	381086ea-f511-4aba-bdf9-71c753dc5077	2026-05-29T01:48:55.869779
12	Arctic Monkeys	arctic monkeys	944116353	7151088	ada7a83c-e3e1-40f1-93f9-3e73dbc9298a	2026-05-29T01:48:55.869779
13	Billie Eilish	billie eilish	848236311	4273090	f4abc0b5-3f7a-4eff-8f78-ac078dbce533	2026-05-29T01:48:55.869779
14	Frank Ocean	frank ocean	786563135	4302551	e520459c-dff4-491d-a6e4-c97be35e0044	2026-05-29T01:48:55.869779
15	Coldplay	coldplay	785537565	9155733	cc197bad-dc9c-440d-a5b5-d52ba2e14234	2026-05-29T01:48:55.869779
16	Linkin Park	linkin park	754904756	6982027	f59c5520-5f46-4d2c-b2c4-822eabf53419	2026-05-29T01:48:55.869779
17	Charli xcx	charli xcx	734132350	3814502	260b6184-8828-48eb-945c-bc4cb6fc34ca	2026-05-29T01:48:55.869779
18	Beyoncé	beyoncé	706936483	6427066	859d0860-d480-4efd-970c-c05d5f1776b8	2026-05-29T01:48:55.869779
19	Travis Scott	travis scott	690776741	3369939	e4a51f17-a57b-47b1-b37b-f552d0f8e9e6	2026-05-29T01:48:55.869779

Figure 9: Sample Silver Parquet output for top artists

```
df_sil_rute = '../datalake_silver/lastfm_top_tracks/lastfm_top_tracks_20260529_014902.parquet'
df_paq = pd.read_parquet(df_sil_rute)
df_paq.head(30)
```

✓ 0.0s Python

	name	name_tokens	duration_sec	playcount	listeners	mbid	artist_name	artist_name_tokens	artist_mbid	ingested_at
0	SWIM	swim	159	198661735	456991	6f33dc05-cdc0-4a2f-8039-e8fed082eec6	BTS	bts	0d79fe8e-ba27-4859-bb8c-2f255f346853	2026-05-29T01:48:55.860499
1	Creep	creep	235	61244353	4114888	012e70cd-4ef6-37bf-8ebf-02d318fe5151	Radiohead	radiohead	a74b1b7f-71a5-4011-9441-d0b5e4122711	2026-05-29T01:48:55.860499
2	Good Luck, Babe!	good luck babe	218	60643840	2100652	1469986c-a292-4d2c-a253-89ed5dc56bf2	Chappell Roan	chappell roan	56a55378-f155-48de-80a5-d80104221267	2026-05-29T01:48:55.860499
3	Pink + White	pink white	290	59944740	2462316	0e360068-12c7-4012-853c-6a6011e0b36a	Frank Ocean	frank ocean	e520459c-dff4-491d-a6e4-c97be35e0044	2026-05-29T01:48:55.860499
4	505	505	305	58803216	3050877	00d0985d-278e-4a72-b959-fc184a1ce990	Arctic Monkeys	arctic monkeys	ada7a83c-e3e1-40f1-93f9-3e73dbc9298a	2026-05-29T01:48:55.860499
5	See You Again (feat. Kali Uchis)	see you again feat kali uchis	180	58283364	2582928	681a3318-d896-4696-8745-0b9aaf642040	Tyler, The Creator	tyler the creator	f6beac20-5dfe-4d1f-ae02-0b0a740aafd6	2026-05-29T01:48:55.860499
6	Body to Body	body to body	189	56037565	403722	6dfdfc61-89ff-451a-a5e9-5319065beae7	BTS	bts	0d79fe8e-ba27-4859-bb8c-2f255f346853	2026-05-29T01:48:55.860499
7	Hooligan	hooligan	182	53925751	379449	a045f252-ea1a-42c3-b7a5-f69979e39394	BTS	bts	0d79fe8e-ba27-4859-bb8c-2f255f346853	2026-05-29T01:48:55.860499
8	2.0	20	169	53377250	349925	1d6222e0-77eb-4047-b4fb-045a0c5636d3	BTS	bts	0d79fe8e-ba27-4859-bb8c-2f255f346853	2026-05-29T01:48:55.860499
9	Mr. Brightside	mr brightside	222	51873262	3711285	0157fd48-a126-4af1-ad1f-4fbb37911b90	The Killers	the killers	95e1ead9-4d31-4808-a7ac-32c3614c116b	2026-05-29T01:48:55.860499
10	FYA	fya	180	51566263	355845	802e86e2-5da4-40c5-a265-933eb64d864b	BTS	bts	0d79fe8e-ba27-4859-bb8c-2f255f346853	2026-05-29T01:48:55.860499

Figure 10: Sample Silver Parquet output for top tracks

8 Challenges and Decisions

This second workshop came with a few decisions that ended up changing how we originally planned things, mostly for the better.

The biggest one was switching from pulling data from a single Last.fm endpoint to using two. At first we were only hitting `chart.getTopArtists`, but we realized quick that having only artist-level data wasn't going to be enough for a solid sentiment analysis.So we added textttchart.getTopTracks

as well, and now each daily run collects both datasets in parallel. It's a small change on paper but it gives us a lot more to work with when we get to the analysis phase.

For Reddit, we also shifted our approach a bit. Instead of scraping one big general music community, we decided to split the ingestion across multiple genre-specific subreddits. The idea is that people in a metal subreddit and people in a pop subreddit are going to talk about the same artist very differently, and that contrast is actually really useful for sentiment analysis. Lumping everything together would've flattened a lot of that nuance.

On the technical side, the main challenge was getting the Bronze-to-Silver pipeline actually working end to end. Building the cleaning functions, the duplication logic, and the schema enforcement took more iteration than expected, but it was necessary to get right since everything in the Gold layer depends on the quality of what comes out of Silver.